

EduMind: An AI-Powered Adaptive Learning Platform with Persistent Learner Modeling

Project Proposal for UCS503P
Thapar Institute of Engineering and Technology

Made by — Mihir Khanna, Parth Khurana, Ojesh Srivastav

21 August 2026

1 Higher-order goal *...secondary framing*

EduMind is an AI-powered adaptive learning platform designed to provide continuous, personalised learning rather than isolated question-answer interactions. The system builds a persistent model of the learner, tracks concept-level mastery and misconceptions, and uses this state to adapt future lessons. The engineering objective is to create a deployable web platform that combines adaptive teaching, a concept knowledge graph, branching exploration, visual learning, retrieval-grounded material and persistent learner progress.

2 Time-to-value *...primary heuristic*

- I shall deliver a vertical slice early — one concept carried end-to-end through onboarding, diagnostic, lesson and progress update within the first iteration — rather than building each subsystem to completion in isolation.
- I shall prioritise fast feedback over breadth, using weekly iterations with CI-backed automation so that every increment is releasable to staging.
- Success in the first iteration is defined by what can be instrumented and measured — cost per session and change in mastery — not by feature count.

3 Problem Statement

Undergraduate students increasingly use general-purpose conversational assistants as informal tutors. Those tools were not designed for instruction, and four failures recur:

- **Statelessness:** the assistant keeps no durable model of the learner. Each session restarts from zero, so instruction cannot adapt to what a student has already mastered or repeatedly gets wrong.
- **Undiagnosed error:** answers are marked wrong without identifying the misconception beneath them. The student learns the correct answer, not the faulty reasoning that produced the incorrect one.
- **Context loss on tangents:** a clarifying side-question either derails the explanation in progress or is asked in a fresh thread, where the lesson's context is lost.
- **Unbounded, unattributed cost:** routing every turn to a single frontier model makes sustained tutoring economically infeasible at scale, and offers no mechanism to attribute or cap spend per learner.

Why this matters now (contextual relevance): published inference prices for hosted language models differ by more than an order of magnitude between capability tiers. The dominant determinant of unit cost is therefore not which model is chosen but the architecture deciding which model handles which task — a systems problem, tractable within a semester.

4 Proposed Solution *...Software Proposition*

4.1 Overview

A web-based adaptive learning platform in which learner state persists across sessions and the teaching experience is coordinated through a central AI orchestration layer:

- Knowledge graph: builds a prerequisite-linked concept map so students can understand concepts and their dependencies.
- Diagnostic module: evaluates the learner through short assessments and records concept-level mastery and misconceptions.
- Adaptive teaching engine: generates lessons conditioned on learner state, selected concepts and uploaded learning material.

- Branch manager: turns side-questions into independent, resumable learning branches without losing the main lesson context.
- Revision scheduler: identifies concepts due for review using stored mastery and retention information.
- Digital learner twin: maintains versioned learner snapshots containing mastery, confidence, misconceptions and learning history.
- Visual and AI workspace: supports explanations, summaries, quizzes, flashcards, mind maps and context-aware visual learning when useful.

4.2 Core workflow

1. The student selects a subject or learning goal; EduMind builds or loads the relevant concept graph and initial learner profile.
2. The student completes a diagnostic assessment, producing concept-level mastery and misconception information.
3. The student opens a concept; the teaching engine retrieves learner state and relevant uploaded material, then streams an adaptive lesson.
4. A side-question can create a real branch for deeper exploration; the student can return to the original learning path without losing context.
5. At session close, the system updates the digital learner twin, records progress and schedules relevant concepts for revision.

4.3 Operational constraints for an educational setting

- The interface shall be responsive and usable on low-to-mid range devices over intermittent connectivity.
- No novel machine-learning research shall be attempted. Hosted models are consumed as commodity services; the contribution is the routing, caching and state architecture around them.
- The system shall be deployable on managed free-tier hosting, so it can be demonstrated without institutional infrastructure.
- Personal data collection shall be minimised: nothing beyond what the learner model requires is stored.

5 Objectives *...specific and measurable*

Primary objective: to design, implement and evaluate a deployable AI learning platform that maintains persistent concept-level learner state across sessions and uses that state to adapt teaching, exploration and revision.

- To implement a central AI orchestration and routing layer through which model-driven tasks are selected and managed — by Week 5.
- To maintain a measurable and bounded model cost per active learning session through model routing, caching and deterministic execution — by Week 12.
- To implement a persistent digital learner twin with versioned snapshots of mastery, confidence, misconceptions and learning history — by Week 10.
- To demonstrate measurable improvement in concept-level diagnostic performance between pre- and post-pilot assessment for at least 15 participants — by Week 13.
- To maintain a CI pipeline that builds, tests and deploys the system to staging on every change, covering core routing, state and learning modules — from Week 3 onward.

6 Solution Approach *...engineering focus*

This is an engineering course project. The emphasis falls on system architecture, correctness and measurement rather than on advancing model capability.

6.1 Non-research stance *...deliberate*

Models are treated as interchangeable commodity endpoints behind a stable internal interface, and no claim of state-of-the-art performance is made. Where a decision could be taken by either a model or deterministic code, deterministic code is preferred: objective grading, revision scheduling and study-order planning are ordinary program logic and incur no model cost.

6.2 System architecture and technology stack

- Frontend / Presentation: Next.js 16, React 19 and TypeScript 5 with App Router, Framer Motion, Recharts, KaTeX and custom canvas-based learning interfaces.
- Backend / Orchestration: Python 3.13, FastAPI and Uvicorn, with LangGraph and Pydantic v2 for stateful orchestration and SSE streaming for incremental lesson delivery.
- AI / Intelligence: GPT-5-mini, Claude-Sonnet-4.5 and Gemini-2.5-Flash accessed through task-based model routing, with SerpAPI available for external information retrieval when required.
- Data / Infrastructure: PostgreSQL 16 with pgvector, SQLAlchemy 2 async and Alembic, with Docker Compose for development and managed deployment using services such as Render, Neon and Vercel.

6.3 The cost-governance mechanism *...central contribution*

AI routing is centralised so that feature code does not directly depend on a specific model. Each task declares the capability it needs and the routing layer selects an appropriate model. High-frequency or deterministic operations can bypass expensive model calls, repeated generations can be cached, and session-level learner synthesis can be batched. This keeps the system responsive and makes model usage measurable and controllable while preserving the flexibility of the adaptive learning experience.

6.4 Data collection and analysis

Instrumentation is designed before implementation, not retrofitted. Each routed call writes a record of task, tier, token counts, latency, cache-hit flag and session identifier; cost per session is computed by joining those records to published token prices. Learning gain is measured within-subject, as the mean paired difference in concept-level score between equivalent-form diagnostics taken before and after the pilot. Every figure derives from a stored timestamp or counter rather than from self-report, so each claim is individually attributable.

6.5 CI/CD and fast delivery enabler

Continuous integration builds the project and runs unit and integration tests on every change, lints the codebase, rejects changes that regress the routing or state modules, and produces a repeatable deployment to staging — so that features can be released frequently and safely within weekly iterations.

7 Evaluation Criterion *...measurable and attributable*

7.1 Primary evaluation metric

Median model cost per active learning session (USD), where an active session is one containing at least one completed lesson or assessment.

- **Target:** median cost per active session of US\$0.05 or less over the pilot window.
- **Attribution:** computed directly from router call records joined to published token prices; no estimation or self-report is involved.

This metric is primary because it is the falsifiable claim on which the architecture rests: a tutoring system that cannot bound its per-session cost cannot be operated continuously, however good its pedagogy.

7.2 Secondary evaluation metrics

- **Learning gain:** mean paired difference in concept-level diagnostic score, pre- versus post-pilot.
- **Cache-service rate:** proportion of generation requests served from cache; target 40% or above.
- **Responsiveness:** 95th-percentile time from lesson request to first streamed token; target 3 seconds or less.
- **Reliability:** availability of staging for login, lesson and assessment during declared pilot hours; target 99% or above.

7.3 Pilot validation plan *...high-level*

- Recruit 15–25 undergraduate volunteers across two subjects.
- Administer a baseline diagnostic before first use and an equivalent-form diagnostic at the close of a two-week usage window.
- Compare pre- and post-distributions within-subject, and extract cost, cache and latency metrics from logs over the same window.
- Collect a 1–5 clarity rating per lesson to contextualise the quantitative results.

8 Scalability *...with foresight*

1. Scalable (theoretically). The system should absorb growth in learners and concurrent sessions by:
 - stateless API design, so backend instances can be added horizontally;
 - indexed queries and pagination over the concept, lesson and branch tables;
 - cohort-keyed caching, whose benefit grows with cohort size — the marginal model cost of an additional student in the same mastery band approaches zero;
 - a reserved vector column in the schema, so semantic retrieval can replace keyword retrieval without migrating application logic.
2. Operationally deployable (practically). The system should run on common hosting: a managed relational database; container- or platform-hosted deployment of the backend, which must be a long-lived process because it holds conversation checkpoints and streams responses; and a CI/CD pipeline targeting staging and production.

9 Engine availability heuristic

A mature set of engines exists for every part of this system, keeping effort on design rather than infrastructure: an asynchronous Python web framework with server-sent-event support; a graph-based orchestration library with a database-backed checkpoint; a hosted multi-model inference API exposing several capability tiers behind one credential; a managed relational database with a vector extension; and a test framework with a CI runner and staging deployment target.

10 Project scope and deliverables *...iteration-friendly*

10.1 Initial deliverable *...first iteration*

- Authentication and learner profile creation; concept knowledge graph seeded for one subject
- Diagnostic assessment with concept-level mastery and misconception write-back
- Adaptive streamed lesson for a selected concept, conditioned on recorded learner state
- Digital learner twin with versioned progress snapshots and a learner dashboard
- Core AI workspace with explanations, summaries, quizzes and learning-support tools

10.2 Subsequent deliverables

- Branch threads with checkpoint forking and resumption
- Uploaded-material ingestion and retrieval-grounded lessons
- Generation caching and task-based AI model routing
- Spaced-repetition scheduler, visual learning layer, knowledge-map exploration and progress/cost reporting

10.3 Explicitly out of scope *...to keep the scope defensible*

Mobile-native clients, institutional LMS integration, multi-tenant administration, offline operation, and any training or fine-tuning of models are excluded. Any addition to §10.2 must displace an existing item rather than extend the list.

11 Timeline *...with a buffer week*

Weeks	Phase	Principal tasks	Deliverable
1–2	Foundation	Requirements; schema design; instrumentation plan; CI skeleton	Approved schema; CI running
3–5	Core slice	Routing layer with call logging; auth; seeding; diagnostic	Instrumented router on staging
6–8	Teaching engine	Streamed lessons conditioned on learner state; branch forking	End-to-end vertical slice
9–10	Cost and state	Caching; revision scheduler; snapshots; learner dashboard	Cache-service rate measurable
11	Pilot	Recruit participants; baseline diagnostic; window opens	Baseline dataset captured
12	Pilot and analysis	Close window; post-diagnostic; extract cost and gain metrics	Results dataset and analysis

Weeks	Phase	Principal tasks	Deliverable
13	Buffer	Reserved for slippage, defect repair and metric re-runs	—
14	Reporting	Final report, documentation and demonstration	Final report and demo

12 Resources *...feasibility*

Material resources. All development tooling is open-source and already available. The system requires hosted model-inference credit, a managed database, and staging and production hosting; free tiers of the latter two suffice for a pilot of this size, and no laboratory or specialised hardware is needed.

Item	Basis	Estimated cost
Model inference credit	Pilot usage bounded by the per-session budget in §7.1	≤ US\$25
Managed relational database	Free tier	Nil
Application hosting (staging and production)	Free tier	Nil
Image-search API (optional visual layer)	Free tier, capped monthly quota	Nil
Total		≤ US\$25

No institutional funding is requested. The inference credit is self-funded and is bounded by the same routing budget the project sets out to evaluate.

13 Risks and mitigations

- **Model availability drift:** a hosted model may be withdrawn or refuse programmatic access mid-semester. The routing table is the only place a model is named, so substituting an equivalent tier is a one-line change; a probe script verifies tier availability at the start of each iteration.
- **Cost target not met:** the median may exceed US\$0.05. Routing table and cache keys are tunable independently of feature code; should the target still be missed, the measured figure is reported as a negative result, since the metric is instrumented either way.
- **Insufficient pilot recruitment:** fewer than 15 volunteers may enrol. Cost, cache and latency metrics do not depend on cohort size and remain valid; the learning-gain claim would then be reported as descriptive rather than inferential.
- **Evaluation ambiguity:** session boundaries and mastery transitions must be unambiguous. Both are defined and fixed in Weeks 1–2, before feature development begins.
- **Free-tier operational limits:** hosted free tiers may sleep or throttle during a demonstration. Staging is exercised from Week 3, and cached lessons with stored state allow a demonstration to proceed without new model calls.
- **Scope growth:** the problem space invites feature accretion. Section 10.3 fixes the exclusions and the Week 13 buffer absorbs slippage, not new scope.

14 Summary

EduMind is a deployable AI-powered adaptive learning platform in which the learner model persists across sessions and directly influences what the student learns next. Its core contribution is the integration of a concept knowledge graph, adaptive teaching engine, branching exploration, visual learning tools, retrieval-grounded material and a versioned digital learner twin within a single web platform. A central orchestration and model-routing layer keeps AI usage measurable and controllable. The project therefore focuses on practical engineering: persistent learner state, adaptive learning workflows, AI orchestration, caching, reliable web delivery and measurable evaluation rather than training a new foundation model.